

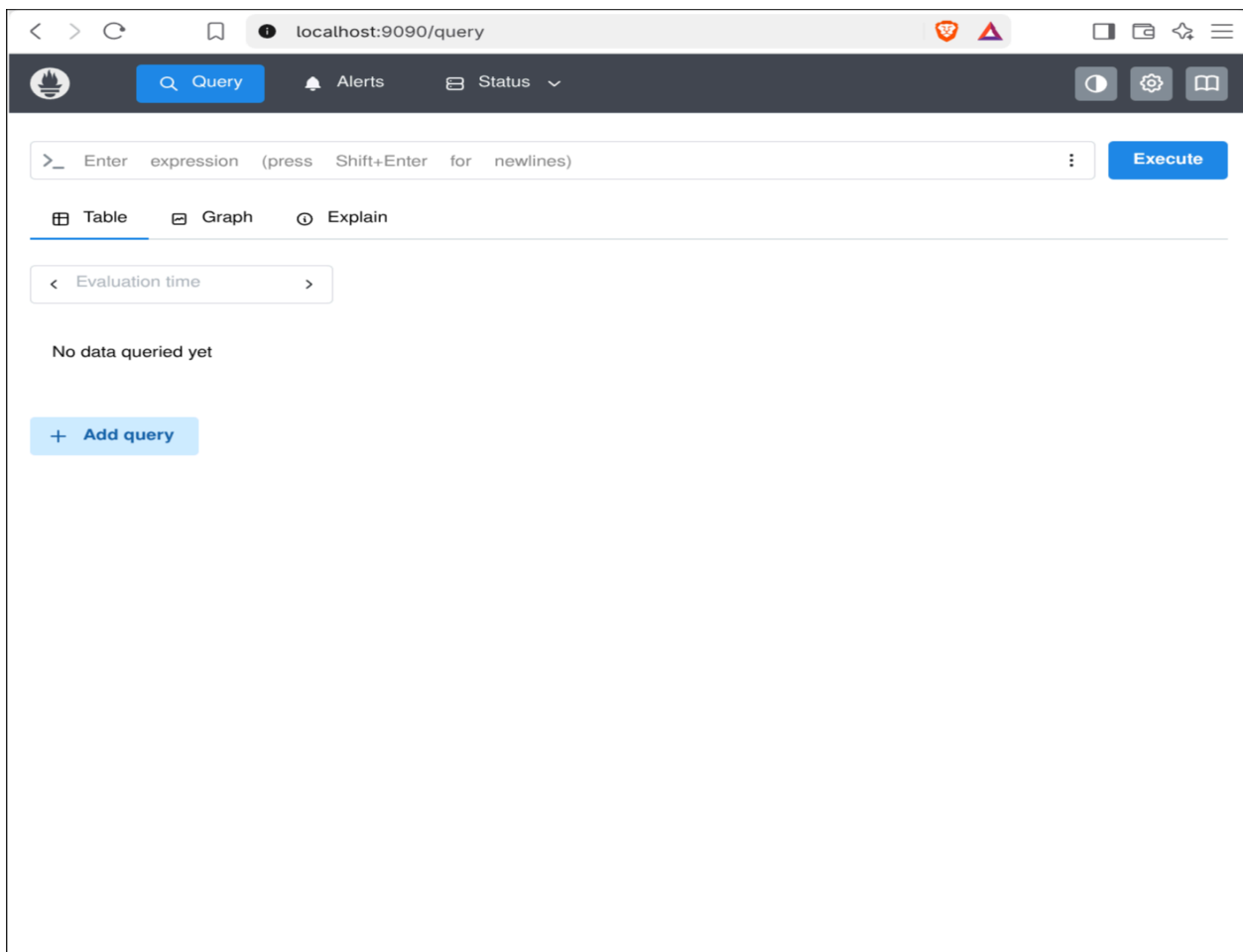
## Задание 1

```
lly@lly ~$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
"prometheus-community" already exists with the same configuration, skipping
lly@lly ~$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "prometheus-community" chart repository
Update Complete. ✨Happy Helming!✨
lly@lly ~$ helm install prometheus prometheus-community/kube-prometheus-stack --namespace monitoring --create-namespace
level=ERROR msg="release name check failed" error="cannot reuse a name that is still in use"
Error: INSTALLATION FAILED: release name check failed: cannot reuse a name that is still in use
lly@lly ~$ kubectl get pods -n monitoring
```

| NAME                                                   | READY | STATUS  | RESTARTS    | AGE  |
|--------------------------------------------------------|-------|---------|-------------|------|
| alertmanager-prometheus-kube-prometheus-alertmanager-0 | 2/2   | Running | 6 (20m ago) | 2d1h |
| prometheus-grafana-74dd9f44b8-mxr5x                    | 3/3   | Running | 9 (20m ago) | 2d1h |
| prometheus-kube-prometheus-operator-d85b5c995-tb84x    | 1/1   | Running | 4 (20m ago) | 2d1h |
| prometheus-kube-state-metrics-599975f55f-gs6cl         | 1/1   | Running | 4 (20m ago) | 2d1h |
| prometheus-prometheus-kube-prometheus-prometheus-0     | 2/2   | Running | 6 (20m ago) | 2d1h |
| prometheus-prometheus-node-exporter-v4ngb              | 1/1   | Running | 3 (20m ago) | 2d1h |

```
lly@lly ~$
```

## Установка Prometheus



## Окно интерфейса

>\_ up{job="web-app-service"}

:

Execute

Table

Graph

Explain

< Evaluation time >

Load time: 17ms Result series: 2

|                                                                                                                                                                             |   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| up{container="web-app", endpoint="http", instance="10.244.0.37:5000", job="web-app-service", namespace="default", pod="web-app-bb677c678-4dlgj", service="web-app-service"} | 1 |
| up{container="web-app", endpoint="http", instance="10.244.0.36:5000", job="web-app-service", namespace="default", pod="web-app-bb677c678-rrz4l", service="web-app-service"} | 1 |

Запрос up отображает информацию о сервисе, созданные поды, эндпоинты, название и namespace

## Задание 2

| Метрика                           | Что измеряет                                                            | Тип       | Единица      |
|-----------------------------------|-------------------------------------------------------------------------|-----------|--------------|
| container_cpu_usage_seconds_total | Суммарное процессорное время, потраченное контейнером с момента запуска | Counter   | Секунды      |
| node_cpu_seconds_total            | Время CPU узла по каждому ядру и режиму                                 | Counter   | Секунды      |
| container_memory_usage_bytes      | Вся занятая контейнером память, включая файловый кэш                    | gauge     | байты        |
| node_memory_MemAvailable_bytes    | Доступная для новых процессов память на узле                            | gauge     | байты        |
| http_requests_total               | Количество обработанных HTTP-запросов                                   | counter   | Запросы (шт) |
| http_request_duration_seconds     | Распределение времени ответа                                            | histogram | секунды      |
| http_requests_total{code=~"5.."}  | Количество запросов, завершившихс                                       | counter   | Запросы (шт) |

|                       |                                                         |       |     |
|-----------------------|---------------------------------------------------------|-------|-----|
|                       | я серверной<br>ошибкой                                  |       |     |
| machine_scraper_error | Проверяет<br>возможность<br>получить<br>метрики         | gauge | 0/1 |
| kube_pod_status_ready | Проверяет<br>возможность<br>пода<br>принимать<br>трафик | gauge | 0/1 |
| kube_pod_status_phase | Проверяет<br>статуса пода                               | gauge | 0/1 |

### Задание 3

| VU  | RPS              | p95          | p99           | error% | cpu%   | Mem mb |
|-----|------------------|--------------|---------------|--------|--------|--------|
| 10  | 9.8652<br>55/s   | =20.55<br>ms | =22.04<br>ms  | 0.00%  | ~0.33% | ~60mb  |
| 50  | 48.938<br>12/s   | =59.37<br>ms | =70.04<br>ms  | 0.00%  | ~1.26% | ~62mb  |
| 200 | 192.40<br>1376/s | =92.67<br>ms | =113.9<br>3ms | 0.00%  | ~5.45% | ~63mb  |



Prometheus выдает 4.75ms

```
execution: local
  script: test-constant.js
  output: -

scenarios: (100.00%) 1 scenario, 100 max VUs, 40s max duration (incl. graceful stop):
  * default: 100 looping VUs for 10s (gracefulStop: 30s)

TOTAL RESULTS

checks_total.....: 1000    97.404473/s
checks_succeeded...: 100.00% 1000 out of 1000
checks_failed.....: 0.00%   0 out of 1000

✓ status is 200

HTTP
http_req_duration.....: avg=17.07ms p(95)=71.35ms p(99)=91.95ms
  { expected_response:true }...: avg=17.07ms p(95)=71.35ms p(99)=91.95ms
http_req_failed.....: 0.00%   0 out of 1000
http_reqs.....: 1000    97.404473/s

EXECUTION
iteration_duration.....: avg=1.01s  p(95)=1.07s  p(99)=1.1s
iterations.....: 1000    97.404473/s
vus.....: 100    min=100    max=100
vus_max.....: 100    min=100    max=100

NETWORK
data_received.....: 189 kB 18 kB/s
data_sent.....: 70 kB 6.8 kB/s

running (10.3s), 000/100 VUs, 1000 complete and 0 interrupted iterations
default ✓ [=====] 100 VUs 10s
```

A k6 71.35ms

Разница в мс:  $71.35 - 4.75 = 66.6$

Разница в процентах:  $66.6 / 71.35 * 100 = 93.3\%$

Разница обусловлена в самом подходе к измерению, k6 считает полный цикл от получения запроса сервером и отправки ответа, а prometheus считает сразу после получения запроса сервером. Т.к. Сервер однопоточный, то и запросы стоят в очереди, поэтому в k6 время запроса дольше.

#### **Задание 4**

1. P95 растёт прямо пропорционально  $v_i$ , т.е. При увеличении  $v_i$  увеличивается и  $p_{95}$ .
2. Нагрузка  $cpi$  тоже увеличивается прямо пропорционально  $v_i$
3. Memory практически не изменяется, но при увеличении  $v_i$  также увеличивается и memory
4. Rps ведёт себя также
5. Ошибки не появлялись
6. При увеличении  $v_i$  сервер ожидаемо деградирует, тк он работает в однопоточном режиме и количество  $v_i$  напрямую влияет на его скорость работы

## Контрольные вопросы

- 1. Что такое мониторинг и зачем он нужен в высоконагруженных системах? Чем мониторинг отличается от нагрузочного тестирования? -**

Мониторинг это сбор метрик системы (cpu, mem, rps и ттд). Он нужен, чтобы наблюдать за метриками системы, отслеживать деградацию и инциденты, следить за производительностью системы. Мониторинг - это наблюдение за системой, а нагрузочное тестирование - искусственно нагружает систему.

- 2. Какую информацию дает k6, а какую Prometheus?**

**Почему их показатели RPS и p95 могут не совпадать?**

- k6 дает информацию со стороны запросов, сколько запросов пришло, сколько упало, сколько запросов приходит за секунду и тд. Прометеус же дает информацию со стороны системы, он показывает метрики ресурсов (cpu, memory), сколько запросов обработал сервер. Rps и p95 могут отличаться из-за разницы в сборе метрик, k6 смотрит со стороны запросов, а prometheus со стороны системы.

- 3. Что такое Prometheus и в чем заключается**

**pull-модель?** - Система мониторинга, собирает метрики, хранит их, отображает графики метрик.

Pull-модель - prometheus периодически делает запрос на /metrics и собирает значения метрик

- 4. Что такое метрика, временной ряд и labels? Зачем**

**нужны labels?** - Метрика ресурсы компьютера (cpu memory disk), а также значения запросов. Временной ряд - значения метрик в определенный момент времени. Labels - пары ключ-значение, прикрепленные к определенной метрике.

- 5. Чем counter отличается от gauge? Приведите**

**примеры. Почему rate() нельзя применять к gauge? -**  
Значение, которое может только возрастать

(http\_requests\_total). Gauge - значение, которое и растёт и убывает

(container\_memory\_working\_set\_bytes) Rate вычисляет среднюю скорость роста counter за интервал, значение должно увеличиваться. Если применить к gauge, то rate может некорректно возвращать значения.

**6. Почему Histogram часто используется для анализа latency в prometheus? Чем Histogram отличается от Summary? Как получить p95 через prometheus? -**

Histogram раскладывает запросы по бакетам, каждый бакет - это counter числа запросов. Histogram в отличие от summary выполняется на самом сервера prometheus, имеет меньшую точность, есть возможность агрегировать данные. P95 можно получить: `histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))`

**7. Что такое PromQL? Как определить количество HTTP**

**5xx ошибок и их долю с его помощью? -** PromQL - язык запросов Prometheus для выборки, фильтрации, агрегации и вычислений над временными рядами. Получение количества 5xx ошибок:

```
sum(rate(http_requests_total{code="5.."}[5m]))
```

**8. Что такое bottleneck? Почему одна метрика**

**(Например, CPU=90%) сама по себе не доказывает наличие bottleneck? -** Bottleneck - это этап или ресурс, который ограничивает производительность всей системы. 90% нагрузка CPU не обязательно говорит о bottleneck, тк сервис может работать и при 90% загрузке процессора, нужно дольше наблюдать за системой, чтоб установить в чем заключается проблема.

**9. Что такое warm-up и saturation point? Почему первые секунды теста стоит игнорировать? -** warm-up -

начальный период теста, когда система еще не вышла

на стабильное состояние. saturation point - уровень нагрузки, после которого RPS перестает расти, а latency и число ошибок резко увеличиваются. Первые секунды теста стоит игнорировать потому что показатели не являются объективными. Они искажают метрики

**10. Чем гипотеза о bottleneck отличается от доказанного результата измерения? Как правильно сформулировать гипотезу по результатам эксперимента?** - гипотеза - это предположение, а доказанный результат - подтвержденный эксперимент. Чтобы правильно сформулировать гипотезу нужно: наблюдение, предполагаемая причина, подтверждение и воспроизводимость.